

When you run the program, you'll see a window with three buttons. Clicking any of the buttons prints a message showing the name of the button that was clicked.

The key idea here is that since we are using three instances of the same `SimpleButton` class, the behavior of each button will be identical. An important benefit of this approach is that any change to the code in the `SimpleButton` class will affect all buttons instantiated from the class. The main program does not need to worry about any details of the inner workings of the button code, needing only to call the `handleEvent()` method of each button in the main loop. Each button will return `True` or `False` to say that it has or has not been clicked.

Building a Reusable Object-Oriented Text Display

There are two different types of text in a pygame program: display text and input text. Display text is output from your program, equivalent to a call to the `print()` function, except it's displayed in a pygame window. Input text is string input from the user, equivalent to a call to `input()`. In this section, I'll discuss display text. We'll look at how to deal with input text in the next chapter.

Steps to Display Text

Displaying text in a window is a fairly complicated process in pygame because it's not simply displayed as a string in the shell, but requires you to choose a location, fonts and sizes, and other attributes. For example, you might use code like the following:

```
pygame.font.init()

myFont = pygame.font.SysFont('Comic Sans MS', 30)
textSurface = myFont.render('Some text', True, (0, 0, 0))
window.blit(textSurface, (10, 10))
```

We start by initializing the font system within pygame; we do this before the main loop starts. Then we tell pygame to load a particular font from the